

1주차

컴퓨터의 본질

계산을 엄청나게 빠르게 하는 기계. 1초당 수십억번

계산의 결과를 수백 GB의 저장장치를 통해 기억

-> 어떻게 계산을 할까? 언어 내부 built-in 계산 + 사용자가 정의하는 계산

★컴퓨터는 사용자가 지시하는 일만 할 수 있다!

프로그래밍 패러다임

선언적 지식 Declarative

사실 fact에 대한 진술

ex) x의 제곱근은 $y^2=x$ 를 만족하는 y이다

결과에 집중. but 어떻게 구현하는지는 말해주지 않음.

절차적 지식 Imperative

방법에 대한 기술

실행 가능한 단계적 명령어의 집합

대부분의 컴퓨터 프로그래밍은 바로 이 절차적 지식을 작성하는 과정

-> Python은 Imperative 언어로 분류되지만, Declarative 속성 또한 다수 포함하고 있음.

절차적 문제 해결

Fact : x의 제곱근은 $y^2=x$ 를 만족하는 y이다

y에 대한 추측값 g로부터 시작

1. g^2 이 x에 '충분히 근접하다면' g를 x의 제곱근이라고 선언하고 중지
2. 아닐 경우 g와 g/x 의 평균을 계산하여 추측값 업데이트
3. 업데이트된 추측값을 사용하여 답을 찾을 때까지 계산과정을 반복

x=16에 대해 g=3을 최초 추측값으로 시작하는 경우의 예시

알고리즘의 구성 요소

일련의 간단한 절차들

각 절차들이 수행되는 시점을 지정하는 제어 과정의 흐름

전체 과정을 언제 멈추어야 하는지 결정하는 수단

알고리즘의 표현 방법

의사코드(Pseudocode), 순서도(Flowchart) 등...

Stored Program

과거 : 특정 계산을 위해 하드웨어를 매번 재설계 (Fixed Program)

현재 : 메모리에 명령어 세트(프로그램)를 저장하여 교체 가능

-> 하나의 하드웨어로 프로그램 무한 반복 실행 가능

Colossus Mark I : WW2에서 독일의 Lorenz암호 해독을 위해 개발함

최신 스마트폰 : 1969 NASA 컴퓨터 전체의 합보다 계산 성능 우수

하드웨어 5대 구성 요소 - 폰 노이만 구조

입력 장치 : 외부 데이터를 컴퓨터로 전달

출력 장치 : 처리 결과를 외부로 표시
CPU : 연산 및 전체 시스템 제어
RAM : 현재 실행 중인 프로그램 임시 저장 aka 주기억장치
보조기억장치 : 데이터를 영구적으로 저장

CPU : 컴퓨터의 두뇌

ALU : 실제계산 + - * / 수행
Control Unit : 제어 장치. 명령어를 해석하고 명령을 수행하기 위해 각 장치에 필요한 신호를 보냄
Register : CPU 내부의 초고속 임시 저장소. 명령어, 데이터 주소, 데이터 contents 등을 저장

다양한 저장 장치

속도 vs 용량 : 빠른 장치는 비싸고, 용량이 작음
SRAM(cache용, 정적, 빠름, 비쌈) vs DRAM(RAM용, 동적, 느림, 저렴)

Register --> Cache(L1,L2,L3) --> RAM --> Secondary Memory(SSD)
CPU는 대부분의 경우 RAM과 직접 대화
프로그램 실행 시 SSD의 데이터가 RAM으로 이동하여 사용되기도 함.

하드웨어의 소형화 & 고속화

진공관 -> 트랜지스터 -> IC(집적회로) -> VLSI(초고밀도 집적회로)
무어의 법칙: 마이크로칩의 트랜지스터 수는 2년마다 2배 증가한다.

왜 파이썬을 배울까?

커뮤니티, 툴킷 큼
배우기 쉬움, 쓰기쉬움 하지만 할수 있는것은 많다.
크로스플랫폼이다. OS 의존적이지 않음
데이터분석, 머신러닝, 웹 개발등등에 적합함.

프로그래밍 언어의 계층

컴퓨터는 01만 이해함. 인간은 2진수로 명령 내릴 수 없음. 자연어와 기계어 사이에 번역이 필요.
고수준 언어(Python) -- 어셈블리어 -- 기계어

번역의 방식 Compiler vs Interpreter

컴파일러 : 소스코드 전체를 한 번에 기계어로 번역. C, C++, Rust, Go
실행 속도가 빠르지만, 개발 사이클이 길다.(프로그램을 만들 때 조금 기다려야 한다)
플랫폼/HW에 종속적

인터프리터 : 한줄씩 읽어서 속도가 느림 Python, JS
개발과 테스트 과정이 빠르고 유연함.
플랫폼/HW와 무관하게 개발/실행이 가능

파이썬 인터프리터의 동작 방식

REPL 모드 : 한줄 혹은 한 블록씩 즉시 실행, 결과가 바뀌면 즉시 출력 창 닫으면 사라짐
스크립트 모드 : 파일 전체를 처음부터 끝까지 실행. print() 있어야만 출력. 파일로 저장됨

REPL == Read-Evaluate-Print-Loop

2주차 변수와 문장

변수란?

: 변수는 컴퓨터 메모리에 저장된 특정한 값(객체:object)을 참조하는 기호화된 이름
변수는 값을 직접 저장하는 것이 아니라, 값이 저장된 위치에 대한 참조(reference)를 갖고 있습니다!

할당문 : 변수의 이름을 정한 후, 할당 연산자를 사용하여 값을 저장.

생성된 객체의 주소를 변수 n에 바인딩

변수이름 규칙 : 길이제한 X, 영문/숫자/_ Snake case로 쓰는게 좋음! 의미를 알수있게. 복수는 복수형으로

리터럴

: 변수에 할당되기 전, 그 자체로 이미 메모리에 생성된 객체

1_000_000 등 언더스코어 인식X 0b1010(2진) 0o12(8진) 0xA(16진) 1.2e3(1.2*10^3)

True False b'hello' (바이너리형)

f-string : 문자열 안에 변수 값 삽입, :을 통해 정밀 포매팅

```
name, score = "Alice", 95.12345
```

```
print(f"이름: {name}, 점수: {score:.2f}") # 출력: 이름: Alice, 점수: 95.12
```

{값.nf} : 소수점 n번째 자리까지 반올림하여 표현

r-string : W을 이스케이프 문자로 해석하지 않고 그대로 출력

```
path = r"C:\Users\Wname\Wdata" # Wn(줄바꿈) 등이 일반 문자로 처리
```

변수의 바인딩

1. 공유 바인딩 m = n
2. 바인딩 변경 m = 400
3. 타입 재바인딩 n = "foo"

Garbage Collection: 참조를 잃은 고아 객체는 가비지 컬렉터가 자동으로 정리

Mutable(가변) vs Immutable(불변) & Caching

Mutable 가변 : 객체 내부를 수정해도 주소는 그대로

```
a = [1, 2, 3]
a.append(4) # 주소(id) 유지
a += [5] # In-place 수정!
a = a + [5] # 주소(id) 변경!
```

sort(), append(), += 등은 내부를 직접 수정(in-place).

Integer Caching : 작은 정수는 재사용됨

```
x1, y1 = 100, 100
x1 is y1 # True (공유함)
x2, y2 = 300, 300
x2 is y2 # False (새 객체 생성!)
```

할당문의 구조

왼쪽 피연산자 : 반드시 유효한 변수여야 함

할당 연산자(=)

오른쪽 피연산자 : literal, 객체, 표현식

letter_count = word_counter = 0 # Chained assignment 연쇄 할당

a, b, c = 2.0, -1.0, -4.0 # Parallel assignment 병렬 할당

문장 vs 표현식

특징	표현식 (Expression)	문장 (Statement)
정의	값을 생성하는 코드	효과를 생성하는 코드
결과	값 (Value)	효과 (Effect)
과정	평가 (Evaluation)	실행 (Execution)
출력	결과 값이 출력됨	값 없음 (None)

5 + 3 # 값: 8 <- 표현식

len("Hello") # 값: 5 <- 표현식

x = 10 # 효과: 변수 생성 <- 문장

import math # 효과: 모듈 로드 <- 문장

연산자

산술 연산 + - * . ** // %

비교 연산 == < > <= >= !=

논리 연산 and or not

비트 연산 & | ^ (xor) ~(not) <<(left) >>(right)

식별 연산 == is(주소가 같니?)

우선순위 : 괄호 > 거듭제곱 > 단항연산자 > 산술(곱>덧) > 비트 > 비교 > 논리

print(-3 ** 2) # -9 (-3**2))

print((-3) ** 2) # 9

print(10 - -3) # 13 (빼셈과 부호)

val = 10 << 1 + 1 # 40 (10 << 2)

res = 5 & 4 + 1 # 5 & 5 -> 5

덧셈이 비트 AND(&)보다 우선

check = True or False and False

print(check) # True

val = not False and False

print(val) # False (True and False)

데이터 타입

수치형 : int, float, complex

시퀀스 : str

논리형 : bool

Casting : int(), float(), str()

Rounding : round(value, 자리수) 자리수까지 반올림

Type Mixing : 더 큰 자료형으로 자동 변환!

복합 데이터타입

시퀀스형 : 순서 有, 인덱싱/슬라이싱 可

리스트, 튜플

비시퀀스형: 순서 無, 키 또는 집합으로 관리

딕셔너리, 집합(set)

동적 타이핑과 타입 힌트

Static Typing(C, Java) 컴파일 시 고정. 안정성 高 <-> Dynamic Typing(Python) 실행시점에 타입 결정. 유연.

Type Hinting : 타입에 대한 참고형 정보. 인터프리터는 실행 시점에 이 힌트 무시

user_name: str = "Alice"

def greet(name: str) -> str: #리턴값 자료형 힌트

return "Hello, " + name

형변환 Type Casting

input()으로 입력받은 것은 항상 str형이다! -> 명시적으로 형 변환을 해 주어야 함.

int(3.9) # 3	실수->정수 변환하면 절삭이 일어남
bool(0) # False	0을 bool로 변환하면 False
bool(10) # True	0이 아닌 값을 bool로 변환하면 True
bool("") # False	빈 문자열을 bool로 변환하면 False
bool("Hi") # True	무언가 들은 문자열을 bool로 변환하면 True

객체의 구성

객체 + 메서드 = .(점 연산자)

객체 : 파이썬의 모든 데이터

```
s = "hello"
s.upper() # 객체 s의 내재된 메서드 upper 호출
```

시퀀스 타입 : 문자열과 리스트 활용

결합 + 반복 * 인덱싱 []

시퀀스 타입의 메서드

결과 반환형 : str

```
s = "hello"
print(s.upper()) # 결과: "HELLO"
print(s) # 원본 유지: "hello"
```

원본 수정형 : list in-place로 원본을 수정하고 아무것도 반환하지 않음

```
L = [3, 1, 2]
print(L.sort()) # 결과: None
print(L) # 수정됨: [1, 2, 3]
```

3주차 조건문과 반복문

if 문의 기본 구조 if-else문 elif문(생략)

공백과 탭 혼용 금지 : IndentationError 날 수 있음.

데이터 타입과 비교 연산

1.1 + 2.2 == 3.3 #False! 소수점을 비교연산 하는 것은 위험하다.
math.isclose(1.1+2.2, 3.3) #권장 방법
문자열 비교 : 유니코드 값 기준 사전적 순서 비교, 대문자가 소문자보다 작음. 아스키 코드!
리스트 비교 : [1, 2] < [1, 3] #True
 [5, 6] > [7, 1] #False 첫 요소인 5, 7을 비교해서 결과값 내보내기

match-case 구조

```
score = 85
match score :
    case s if s >= 90 : print('A')                      # s는 더미 변수
    case s if s >= 80 : print('B')
    case _ : print('F')                                      # 위에서 일치하는 조건이 하나도 없을 때
```

for 문과 Iterable

Iterable : 요소를 한 번에 하나씩 반환할 수 있는 객체.

리스트, 튜플, 문자열, 딕셔너리 등..

★파이썬에서 for문은 iterable 객체 내부 요소에 “직접 접근”하는 방식을 따름.

Iterator 프로토콜

1. 준비 : `iter()`을 호출하여 이터레이터 생성
2. 반복 : `next()`을 호출 시마다 값을 반환하고 포인터를 이동
3. 종료 : `StopIteration` 신호를 감지하여 자동으로 종료함.

range() 함수

특징 :

지연 평가 : 메모리 효율을 위해 필요할 때 하나씩 값을 계산

불변 시퀀스 : 데이터 전체를 메모리에 올리지 않는 “불변 시퀀스” 객체임

`range`는 그 자체로 리스트가 아니고 `range` 객체임. 메모리를 거의 차지하지 않음. 안전하게 반복 가능.

while 문

무한루프 주의해야 함

`break` : 루프 탈출

`continue` : 루프 건너뛰기

for else, while else 문 : 중간에 `break`를 만나지 않고 수행이 완료되면 `else` 블록이 실행됨

for item in data:

if item == target: break

else: print("Target not found!")

while count < 3:

if error_occurred: break

count += 1

else: print("Successfully completed")

무한 루프

1. 의도치 않은 무한루프.

`while`에서 조건식을 잘못 써서 항상 `True`인 경우

2. 의도적인 무한루프

`input`을 원하는 것을 입력할 때까지 입력하는 경우

for 루프 주의할 점 1

numbers = [2, 4, 6, 8]

for number in numbers:

#원본 데이터를 직접 순회하면

if number % 2 == 0:

numbers.remove(number)

#여기서 2를 삭제할 때, 반복이 어긋남.

대안!

for number in numbers[:]: # [:]로 사본 생성

사본을 생성해서 하면 원본을 건드리지 않음.

if number % 2 == 0:

아니면 그냥 `while`문을 써라.

numbers.remove(number)

for 루프 주의할 점 2

for name in names:

name = name.upper() # 원본은 그대로 유지됨. 원본 리스트에는 아무런 변화가 없음.

for i, name in **enumerate**(names):

#데이터 직접 수정할대는 `enumerate()` 사용하기!

names[i] = name.upper() # 인덱스를 사용하여 원본 리스트에 직접 접근

for 루프 주의할 점 3

루프 변수는 종료 후에도 메모리에 남아 값을 유지한다.

재사용 시 주의가 필요함.

```
for i in range(3):
    pass
print(i) # 출력: 2 (변수가 스코프 내에 생존)
```

데이터의 불확실성. 예상치 못한 타입이 섞여 있으면 루프가 도중에 중단될 수 있음.

```
data = ["10", "error", "30"]
for d in data:
    if d.isdigit():
        print(int(d) * 2) # 안전한 처리를 위해 isdigit()을 if문으로 검사해야 함.
```

for vs while

모든 for 문은 while문으로 바꿀 수 있다.

역은 성립하지 않음.

for 문은 중간에 리스트 길이를 바꾸기 어려움
while문은 가능!

```
colors = ["red", "blue", "yellow", "green"]
while colors:
    color = colors.pop(-1) #맨 뒤를 pop!
    print(f"Processing: {color}")
```

비교 요소	for 루프	while 루프
제어 방식	순회 (Traversal)	조건 제어 (Condition)
반복 횟수	유한 (명확)	가변 (미정)
카운터	자동 내장	수동 초기화/증감
상호 변환	while로 대체 가능	for로 대체 불가능*

제공된 알고리즘 구현

Walrus Operator : `:=` 값을 변수에 할당함과 동시에 그 값을 식의 결과로 반환함.

```
import math
x_number = float(input("Target number: "))
sqrt_guess = float(input("Initial guess: "))
steps = 0
while True:
    is_close = (abs_err := abs(sqrt_guess**2 - x_number)) < 1e-12 # ㅎ

    if not is_close:
        # 제공된 업데이트 로직 (뉴턴의 방법 방식)
        sqrt_guess = (sqrt_guess + x_number / sqrt_guess) / 2.0
        steps += 1
        print(f"{steps} steps... Error: {abs_err:.10f}")
    else:
        # 오차가 충분히 작아지면 루프 탈출
        break
print(f"Calculation completed. Result: {sqrt_guess:.16f}")
```

문자열 제어 - 모음 제거

```
text = input("문장 입력: ")
vowels = "aeiouAEIOU"
result = ""
```

```
for char in text:
    if char not in vowels:
        result += char
```

```
print(f"결과: {result}")
```

문자열 제어 - 모음 위치 탐색

```
for idx, char in enumerate(text):
    if char.lower() in "aeiou":
        print(f"모음 '{char}' 발견! 위치: {idx}")
```

4주차 함수

DRY 원칙 : Don't Repeat Yourself 코드의 중복을 제거하는 원칙 -> 함수로 만들어라

콜 스택 : 리턴 주소, 지역 변수 등을 저장하는 스택 LIFO

파이썬은 Call by **object reference** 객체의 참조를 매개변수에 바인딩. 객체의 가변성에 따라 달라짐
->int, **str** 같은 immutable은 함수 내부에서 수정해도 **안 바뀜**. (전역, 지역변수처럼)
->**list** 같은 mutable은 함수 내부에서 수정하면 **바뀜**

데이터 보존

```
def func(data):
    new_data = data.copy() #안전하게 얇은 복사
    new_data.sort()         #안전

func(raw_data[:])          # 슬라이싱으로 얇은 복사
```

만약 **중첩 구조라면** 중첩 리스트, 딕셔너리 등등...
그냥 .copy()대신 .deepcopy()를 써야 한다.

네임스페이스 : 변수 이름과 객체의 매핑을 관리하는 공간. 함수가 호출될때마다 Local Namespace가 생성된
->**캡슐화**(함수 내외부의 변수가 섞이지 않음) **가능!**

변수 우선순위

Local -> Enclosing(중첩에서 상위) -> Global -> Built-in(파이썬 내장함수)

Shadowing : 이름이 같을 경우 가리는 현상. e.g. Global 변수 가림, Built-in 함수 가림(매우 위험!)

global 키워드

함수 내부에 global [변수이름]을 쓰면 바깥에 있는 전역변수를 사용할 수 있음.

중첩 함수와 nonlocal

nonlocal 키워드 : 상위 함수의 변수를 수정할 때 사용

★인자 전달 방식★

1. Positional Arguments

```
def greet(name, age):  
    printf(f"{name} is {age}")  
  
greet("Alice", 25)
```

2. Keyword Arguments

```
greet(age=25, name="Alice")           #이름 명시. 가독성 ↑
```

-> 만약 혼용 시 Positional이 먼저 와야 한다.

Default Argument 기본인자에서 문제점

```
def add_item(item, cart=[]):  
    cart.append(item)  
    return cart
```

```
print(add_item('A'))           # ['A']  
print(add_item('B'))           # ['A', 'B'] 등으로 누적
```

가변인자

*args	몇 개 올지 모른다! Tuple 로 묶어서 함수 안으로 들어옴. 수정 불가.	* 기호를 씀
**kwargs	kwargs. key:value로 묶여 함수 안으로 들어옴. dict 로 묶여서 들어옴	**기호를 씀

인자 순서는 일반인자 -> * -> ** 순으로 해야 함. 어기면 에러.

여러개 값 반환

```
return min(nums), max(nums)           #tuple로 반환
```

```
low, high = get_stat([1,10,5])
```

변수 교환

```
a, b = 10, 20  
a, b = b, a   (파이썬에서는 정상 동작!) (20,10)이라는 튜플이 생성되고 언패킹 됨.
```

반환값 무시

```
_, max_val = get(stats[1,2,3])         #언더바를 써놓으면 반환값 무시
```

Lambda 함수

-> 29페이지 예시 참조

```
def 없이 함수 정의   lambda x, y : y+ x
```

First-Class Object 일급객체 : 파이썬에서 함수는 '데이터와 같다' -> 변수에 할당, 인자로 전달, 결과 반환 등 可

```
num = [1,2,3,4]  
sq = list(map(lambda x:x**2, nums))  
even = list(filter(lambda x:x%2==0, nums))  
is_odd = lambda x:x%2!=0  
odd = list(filter(is_odd,num))   #이렇게 함수를 객체에 담아서도 전달할 수 있다!
```

Docstring

""" """ 코드 설명서. help()함수로 조회 가능
함수 기능, 인자, 반환값 등 써 놔야 함

좋은 함수란?

단일 책임 원칙 : 함수는 한 가지 일만 해야 함
적절한 길이 유지. 너무 길면 안됨
이름을 의미있게 짓자

5주차 시퀀스 자료구조

시퀀스 Sequence : 순서가 있는 데이터 구조

예시 : list, tuple, str
특징 : indexing, slicing, iteration 가능
연산 : len(), in, +, * 등

리스트는 heterogeneous한 데이터를 넣을 수 있음. 동적 배열임. O(1) 접근 보장

슬라이싱 slicing

기본 문법

L[start:stop] start는 포함, stop은 포함하지 않음

확장 문법

L[start:stop:step] step이 음수이면 역방향 **[::-1]**은 전체 뒤집기

len(seq) : 길이
x in seq : 포함 여부
seq + seq : 연결
seq * n : 반복
seq.index(x) : x의 인덱스는?

가변 시퀀스 vs 불변 시퀀스

가변 mutable

list : 객체 내부를 직접 수정 가능

불변 immutable

tuple, str : 직접 수정 불가. 변경하려면 새 객체를 만들어야 함 +를 통해 새로운 객체 만들기는 가능!

리스트 메서드

.append(x)	맨 뒤에 요소 1개 추가	L.extend(4)
.extend(iterable)	iterable의 요소들을 풀어서 뒤에 추가	L.extend([4,5])
.insert(i, x)	i 위치에 삽입. 앞쪽 삽입일수록 느림	L.insert(3,5) #인덱스 3에 5 넣기
remove(x)	처음 만나는 x 하나 삭제. 없으면 ValueError	
.pop(i)	삭제 + 반환 인덱스 없으면 맨 뒤 .pop()	i번째 요소 pop!
.clear()	전체 삭제	

리스트 정렬과 반전

원본 직접 수정. 반환값이 없다!	L.sort()	L.reverse()
새 결과 생성. 반환값에 저장됨	sorted(L)	reversed(L)

참조와 복사

<code>data.copy()</code>	<code>data[:]</code>	바깥 구조만 복사
<code>copy.deepcopy(data)</code>		원본 데이터를 보존하고, 완전 복사

지연 평가(제너레이터)

`yield` : 리턴하지 않고 순차적으로 결과값만 함수 밖으로 전달
`with open` : 제너레이터 방식으로 동작. 메모리 점유율 일정하게 유지 가능

메모리 복사 비용을 $O(n)$ 이 아닌 $O(1)$ 로 유지

List Comprehension

리스트 안에 조건문이나 반복문 넣어서 사용하기

```
def is_consonant(letter):  
    vowels = "aeiou"  
    return letter.isalpha() and  $\neg$  letter.lower() in vowels  
  
text = "Ted the Rocket"  
result = [c for c in text if is_consonant(c)]  
# 결과: ['T', 'd', 't', 'h', 'R', 'c', 'k', 't']  
  
scores = [85, 42, 78, 59, 92]  
# 60점 이상이면 'Pass', 아니면 'Fail'  
grades = ["Pass" if s >= 60 else "Fail" for s in scores]  
# 결과: ['Pass', 'Fail', 'Pass', 'Fail', 'Pass']  
# 모든 데이터의 '상태'를 변환하여 보존
```

데이터의 개수가 줄어들어야 한다면?	<code>if</code> 를 뒤에 쓰세요.
개수는 유지하고 형태만 바뀌어야 한다면?	<code>if-else</code> 를 앞에 쓰세요.

튜플

순서가 있고 수정 불가능한 **immutable** sequence
원소 1개짜리 튜플은 반드시 콤마 필요 : (3,)

Parallel Assignment

우변이 2개면 튜플로 패키징되어서 전달됨.

```
def get_gcd(a, b):  
    while b != 0:  
        a, b = b, a % b  
    return a  
# 실행 예시  
print(get_gcd(1220, 516)) # 결과: 4  
# Tuple Magic: 임시 변수 없이 두 상태를 동시에 교체
```

함수와 튜플

`enumerate()` : (index, value) 튜플 생성

문자열 str

immutable인 시퀀스이다
문자열 업데이트 할때마다 새로운 리터럴(주소) 할당
문자열을 '+'로 더하면 복잡도가 $O(n^2)$ 이 된다.

join()

-> `"".join(iterable)` 을 쓰는 것이 선형 시간에 가능
`words = ["Python", "is", "Powerful"]`
`print(" ".join(words))` # "Python is Powerful"
`print("-".join(words))` # "Python-is-Powerful"

<code>split()</code>	구분자로 나누어 list 반환
<code>strip()</code>	양끝 공백/특정 문자 제거
<code>replace(old, new)</code>	문자열 교체, 새 문자열 반환

filter()가 안 보일 때

```
clean = "".join(filter(str.isalnum, target))
print(clean)
# 결과: "RaceCar"
```

시퀀스 뒤집기

<code>list.reverse()</code>	in-place 원본 수정
<code>new=reversed(s)</code>	원본 유지. new에 저장됨

실수 주의!

문자열이나 튜플은 직접 수정이 불가능하다!

```
s= "Python"
s[0]="J"      # 불가능!
s = "J"+s[1:] # 가능
```

`len(word)`등으로 C언어처럼 순회하지 말아라! 직접 word라는 리스트로 `for i in word` 등으로 써야 함

6주차 딕셔너리, 세트, 재귀

딕셔너리 Dictionary

key와 value의 쌍으로 이루어진 mapping 자료구조	검색 속도 O(1) 시간복잡도
dictionary는 key로 접근	list는 정수 index로 접근

<pre># 1. 튜플 쌍으로 선언 pairs = [("A", 1), ("B", 2)] d1 = dict(pairs) # {'A': 1, 'B': 2}</pre>	<pre># 3. fromkeys() Method users = ["Lee", "Kim", "Park"] d3 = dict.fromkeys(users, 0) # 모든 키의 초기 점수를 0으로 설정</pre>
<pre># 2. zip() Function keys = ["id", "pw"] vals = ["admin", "1234"] d2 = dict(zip(keys, vals))</pre>	<pre># 4. Keyword Arguments d4 = dict(name="DJ", age=20)</pre>

요소 접근 및 안전한 조회

<code>d[key]</code> 하면 에러	-> 객체.get(key, default)
<code>setdefault(key, default)</code>	누락된 데이터를 default라는 값으로 바꿔줌

'key' in d bool값으로 확인

요소 삭제 및 업데이트

- `.pop(key)` : 키 삭제 후 그 값을 반환
- `.popitem()` : 마지막 쌍을 튜플로 반환 후 삭제
- `.update(other)`: 다른 딕셔너리와 병합. 중복된 키는 기존 값을 덮어씀
- `.clear()` : 초기화. 전부 삭제

`.keys()` : 모든 키 목록
`.values()` : 모든 값 목록
`.items()` : 키, 값 튜플쌍

-> 리스트를 복사하지 않고, 원본을 투영함. 원본이 바뀌면 뷰 결과도 실시간 반영

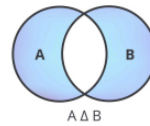
`for k, v in d.items()` # 딕셔너리 순회 시 권장되는 방법

`s_items=sorted(scores)` # 키 값을 기준으로 정렬. 원본 유지.
`s_items=sorted(scores.items(),key=lambda x:x[1])` # `x[1]`에 들어있는 값 기준으로 정렬할 수도 있음.
-> sorted하면 답은 리스트로 나온다! 딕셔너리 아님

딕셔너리 뷰와 연산

`keys()`, `items()`는 집합처럼 동작함. 따라서 수학적 연산 지원

& 교집합 AND | 합집합 OR $\wedge$ 공통 차집합 - 차집합 빼기



```
d1 = {"a": 1, "b": 2}
d2 = {"b": 2, "c": 3}
common_keys = d1.keys() & d2.keys() # 1. 공통 키 추출 (Intersection)
print(common_keys) # {'b'}
only_d1 = d1.keys() - d2.keys() # 2. d1에만 있는 키 (Difference)
same_items = d1.items() & d2.items() # 3. K, V가 모두 같은 항목 추출
print(same_items) # {'b', 2}
```

딕셔너리의 순환

```
scores = {"Alice": 85, "Bob": 90}
for k in scores: #방법 1
    print(k, scores[k])
```

```
for name, val in scores.items(): #방법2 items() 사용
    print(f"{name}: {val}")
```

```
total = sum(scores.values()) #값 필요 없을때 그냥 values만 가지고 sum도 가능
```

딕셔너리 컴프리헨션

```
orig = {"A": 1, "B": 2} # 1. Swapping (Key-Value 반전)
rev = {v: k for k, v in orig.items()}
print(rev) # 결과: {1: "A", 2: "B"}
```

```
scores = {"Lee": 95, "Kim": 70} # 2. Filtering (조건부 추출)
high = {k: v for k, v in scores.items() if v >= 80}
print(high) # 결과: {'Lee': 95}
```

```
langs = ["Python", "Java"] # 3. Data Transformation (데이터 가공)
len_map = {s: len(s) for s in langs}
print(len_map) # 결과: {'Python': 6, 'Java': 4}
```

세트

중복불허, 순서없음, mutable(가변성) 인덱싱, 슬라이싱 불가.

```
s = {1, 2, 2, 3, 3}          # 1. Uniqueness: 중복된 값은 자동으로 무시됨
print(s)                    # 결과: {1, 2, 3} (중복 제거)
```

3. Hashable Elements: 다양한 타입을 가질 수 있지만, **Immutable(불변)** 객체만 가능

list는 요소로 불가능!

```
mixed = {1, "Python", (1, 2)}
print(mixed)
```

세트의 생성

CODE: SET INITIALIZATION

```
s1 = {1, 2, 2, 3}          # 결과: {1, 2, 3} (중복 자동 제거)
s2 = set([1, 2, 3])        # 리스트를 세트로 변환
s3 = set("apple")          # 결과: {'a', 'p', 'l', 'e'} (문자열 내 중복 제거 및 분해)
```

빈 세트를 만들 때는 반드시 **set()** 함수를 호출해야 합니다. 아니면 디셔너리가 됨!

```
empty_set = set()
```

리스트에서 중복을 제거하고 싶을 때 사용하는 파이썬 표준 관례(Idiom)

```
list_obj = [1, 2, 2, 3, 3, 3]
unique_list = list(set(list_obj))          list(set(리스트이름)) 하면 중복 제거!
print(unique_list)                        # 결과: [1, 2, 3]
```

요소 추가 삭제

```
s = {1, 2}
s.add(3)          # 1. 추가 (Add & Update)
s.update([4, 5])  # 여러 요소(Iterable)를 한꺼번에 추가 -> {1, 2, 3, 4, 5}
```

2. 삭제 (Remove vs Discard)

```
s.remove(1)       # 요소 '1'을 정상적으로 삭제
s.remove(99)      # 존재하지 않는 요소를 지우려 하면 KeyError 발생! (주석 처리)
s.discard(99)     # 요소가 없어도 에러 없이 안전하게 무시 (권장 방식)
```

3. 임의의 제거 (Pop) 세트는 순서가 없으므로 '무작위' 요소 하나를 반환하고 제거합니다.

```
val = s.pop()
s.clear()          # 모든 요소를 제거하여 빈 세트 set()로 변환
```

집합 연산

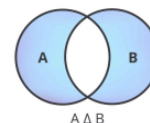
```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # 결과: {1, 2, 3, 4, 5}
print(a & b)  # 결과: {3}
print(a - b)  # 결과: {1, 2}
print(a ^ b)  # 결과: {1, 2, 4, 5}
```

OR

AND

빼기

대칭 차집합



세트 컴프리헨션

리스트 컴프리헨션과 동일하지만 {} 기호를 사용

```
word = "abracadabra"
chars = {c.upper() for c in word}
print(chars)  # {'A', 'B', 'R', 'C', 'D'}
```

재귀 Recursion

```
def factorial(n):  
    if n == 1:                                # 1. Base Case  
        return 1  
    return n * factorial(n - 1)                # 2. Recursive Step  
  
print(factorial(4))                            # Output: 24
```

재귀 호출을 하면 스택 프레임이 생성되어 메모리에 쌓임. LIFO!

재귀와 반복 비교

재귀 : 수학적 직관성. 문제 정의를 그대로 구현 가능. 코드 압도적 간결
반복 : 스택 호출 오버헤드 없음. 속도 빠름 안정적임. 스택오버플로우 없음

피보나치

```
def fib(n):  
    if n <= 1:                                # 1. Base Case (종료 조건)  
        return n  
    return fib(n-1) + fib(n-2)                # 2. Recursive Step (자기 복제)
```

피보나치 Memoization

```
memo = { 0 : 0 , 1 : 1 }  
def fib(n):  
    if n in memo:  
        return memo[n] # 이미 계산된 값 활용  
    memo[n] = fib(n- 1 ) + fib(n- 2 )  
    return memo[n]  
print(fib( 50 )) # 순식간에 계산 완료
```

실행시간 측정 -> 메모이제이션을 하면 속도가 훨씬 빠르다!

7주차 파일과 예외처리

파일 열기 닫기

```
f = open("result.txt", "w", encoding="utf-8")    # 1. 파일 열기 (쓰기 모드)  
scores = [90, 85, 77, 92]                        # 2. 실전 데이터 처리 (리스트를 문자열로 변환하여 기록)  
for i, s in enumerate(scores):  
    line = f"Student {i+1}: {s} points\n"  
    f.write(line)  
f.close()                                         # 3. 명시적 종료 (누락 시 데이터가 저장되지 않을 수 있음)
```

with 문

```
with open("data.txt", "r") as f:                # 블록(Indent)을 벗어나는 순간 f.close()가 자동으로 호출됨  
    content = f.read()  
    print(content)
```

f.read() 파일 전체를 거대한 문자열로 반환
f.readline() 한 줄씩 반환
f.readlines() 1차원 리스트로 반환. 각 줄이 요소

파일모드

'w' : 모두 삭제하고 새로 작성. 없으면 생성
'a' : append 유지한채로 끝부터 데이터 추가
'x' : 배타적 생성. 파일이 있으면 중단, 에러!

덮어쓰기 해버림. 보안 不好

강력한 보안

텍스트 vs 바이너리 모드

텍스트 : 데이터를 문자로 해석. encoding="utf-8" 명시 권장

줄바꿈 문자 \n으로 자동변환

바이너리 : 데이터를 바이트 단위로 취급

CSV 파일 처리

7주차 p13

경로 관리와 OS 모듈

```
import os
cwd = os.getcwd()           # 현재 작업 디렉토리 확인
files = os.listdir(".")     # 디렉토리 내 파일 목록 리스트업
os.mkdir("results")         # 폴더 생성 및 파일 삭제
os.remove("temp.txt")
```

절대경로 vs 상대경로

절대경로 : 루트에서부터 전체 경로

상대경로 : cwd 기준 . 현재 디렉토리 .. 상위 디렉토리 -> 험업할땐 상대경로!

mkdir makedirs() 중간 경로까지 한번에 rename(파일, "이름") remove() rmdir()

현대적인 기법 - OS

os.scandir() : 고속 탐색 listdir보다 빠르다.
pathlib.Path : 객체지향 경로 path를 객체로 취급.

구문오류 vs 런타임 오류

Syntax Error : 해석 단계에서 발생. 틀리면 프로그램은 한줄도 실행되지 않음
콜론 누락, 들여쓰기 잘못, 괄호 불일치

Runtime Error : 실행 도중 발생. 그 전까지는 정상적으로 실행됨.
0으로 나누기, 형변환 오류, 존재하지 않는 주소

예외 계층 구조

모든 예외(Exeption)은 객체다.

AttributeError : 존재하지 않는 속성/메서드 호출

ImportError : 모듈 로드 실패

OSError : 시스템 관련 오류

RuntimeError : 실행 오류

UnicodeError : 인코딩/디코딩 오류

예외처리 기본 문법

try: 예외가 발생할 가능성이 있는 코드를 실행
except: 예외 발생시 대비책
as: 예외가 발생한 객체의 메시지를 변수로 받아서 확인!


```

try:
    n = int(input("나눌 숫자를 입력하세요: "))
    result = 10 / n
    print(f"결과: {result}")
except ZeroDivisionError as e:
    print("오류: 0으로 나눌 수 없습니다.")
    print(f"상세 메시지: {e}")
except ValueError:
    print("오류: 숫자만 입력 가능합니다.")

print("프로그램이 종료되지 않고 계속 실행됩니다.")

```

-> 무차별적으로 예외처리를 하면 안됨. 구체적인 예외 명시해야 함

예외처리의 완전한 구조

```

try:
    f = open("data.txt", "r")
except FileNotFoundError as e:
    print(f"오류: {e}")
else:
    # 파일 열기에 성공한 경우에만 데이터 읽기
    content = f.read()
    print(content)
finally:
    # 결과와 상관없이 어쨌든 실행
    if 'f' in locals() and not f.closed:
        f.close()
    print("Resources cleaned up.")

```

raise 의도적인 에러 발생

```

def check_status(is_active):
    if not is_active:
        raise ServiceError(500, "서비스 비활성화")

```

LBYL vs EAFP

LBYL : Look Before You Leap	둘다리고 두들겨보고 건너자
조건을 먼저 검사하고 실행	
EAFP : Easier to Ask Forgiveness than Permisson	허락보단 용서가 빠르다
실행해보고 안되면 Error try-except	